MITS Gwalior
Campus Walk

Pygame + NumPy se bana 3D walkthrough

Poora code, detail me, diagrams ke saath

Raycasting | Floor casting | SDF roads | Sky panorama | Building | Trees

Contents

1. Badi tasveer aur draw order
2. Setup aur settings
3. Map banana (grid, coordinate, roads)
4. Building ke numbers
5. Ground texture aur bake
6. SDF (smooth roads + collision)
7. Sky panorama
8. Floor casting
9. Walls (raycasting)
10. Building (ray-box)
11. Ped (billboard sprites)
12. Vignette aur minimap
13. Movement aur collision
14. Main loop
15. Sudhar ke sujhav
16. Chalane ka tareeka
17. 30 second pitch aur glossary

1. Badi tasveer

Ye **asli 3D nahi hai**. Ek **2D map** hai jise maths ki trick se 3D jaisa dikhaya gaya hai. Koi 3D engine ya GPU use nahi hua: sirf **pygame** (window aur drawing) aur **numpy** (tez maths).

Har frame me cheezein ek ke upar ek draw hoti hain. Isse **Painter's Algorithm** kehte hain: pehle sabse door/peeche wali cheez, phir paas wali upar se. Naya draw purane ko dhak deta hai.

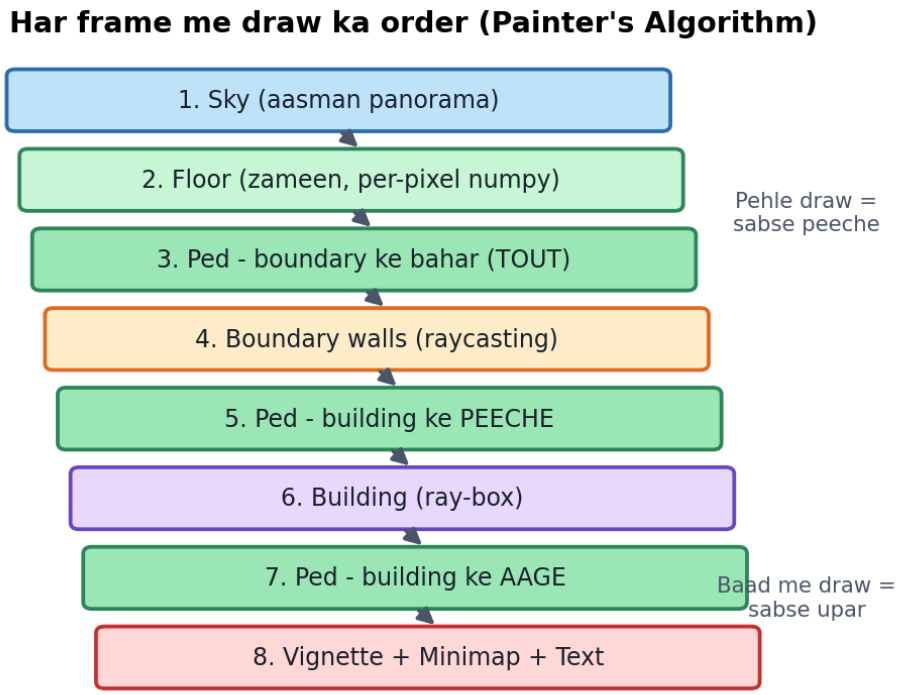


Figure 1: Draw ka order. Number 1 sabse pehle, 8 sabse baad me.

2. Setup aur settings

Variable	Value	Matlab
W, H	1200, 800	Window ka size
HALF	H // 2	Horizon line. Isse upar aasman, neeche zameen
FLOOR_SCALE	2	Floor aadhe resolution par banta hai, phir bada hota hai. 1 = sharp par slow
RAYS	300	Wall ke liye 300 vertical columns. COL = W // RAYS = 4 pixel har column
FOV	pi/3 (60 deg)	Player ka dekhne ka angle
SPEED, RUN, TURN	0.07, 0.14, 0.045	Chalne, daudne, ghumne ki speed
BOUND_H	0.45	Boundary wall ki unchai
MAXD	45	Isse door sab fog me gayab

3. Map banana

3.1 Grid

`grid[row][col]` ek 2D list hai. Har cell ek character:

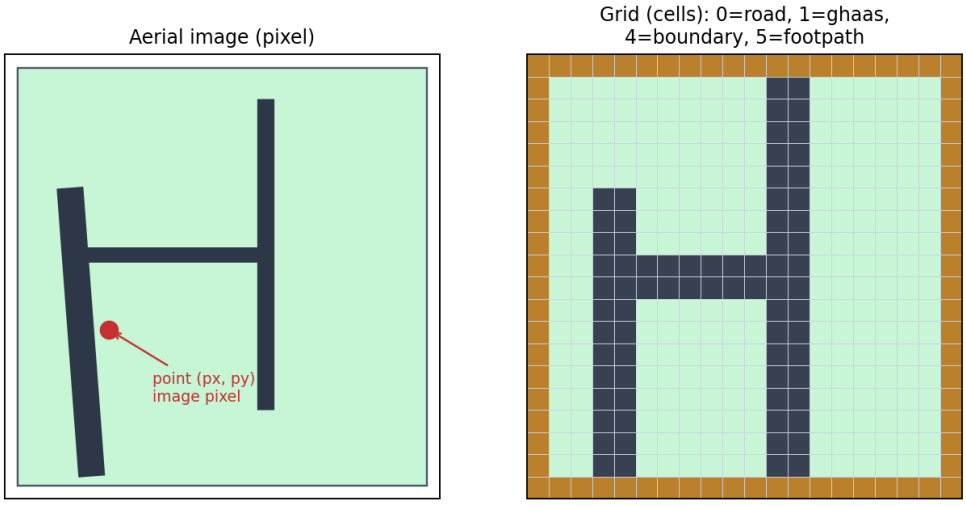
- `"0"` = road
- `"1"` = ghaas
- `"4"` = boundary wall
- `"5"` = footpath

3.2 Pixel se cell (sabse zaroori formula)

Roads ke points campus ki **aerial image ke pixel** me diye hain. Unhe grid ke cell me badalna padta hai.

```
cell_x = (pixel_x - X0) * K / S
cell_y = (pixel_y - Y0) * K / S
```

- **S = 10**: image ke 10 pixel = 1 cell
- **K = 3**: campus lambai-chaudai me 3 guna bada, par road ki chaudai wahi rehti hai
- **X0, Y0**: image me map kahan se shuru hota hai (left ki khali ghaas aur upar ka hissa kaat diya)



cell_x = (pixel_x - X0) * K / S **cell_y = (pixel_y - Y0) * K / S**

Figure 2: Image ke pixel se grid ke cells.

3.3 chaikin(pts): kone gol karna

Har seedhi line ke do point ke beech me **25%** aur **75%** par naye points banata hai, aur ye **4 baar** karta hai. Sharp kone dheere-dheere gol curve ban jaate hain. Pehla aur aakhri point wahi rehta hai.

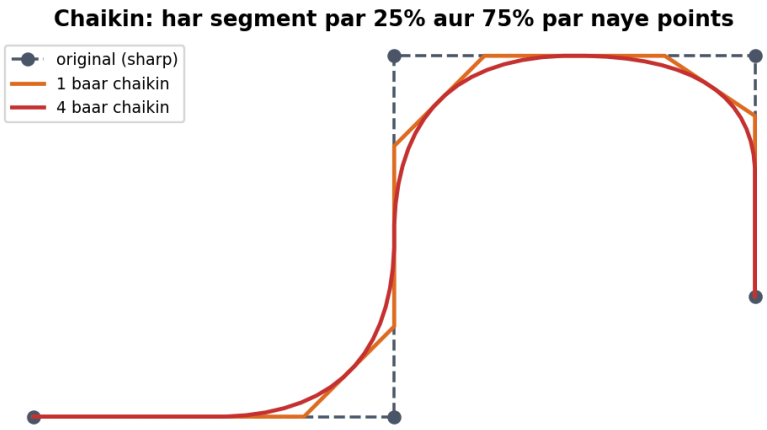


Figure 3: Chaikin smoothing. Jitni baar, utna smooth.

3.4 road(pts, w1, w2)

- Points ko smooth karta hai (`smooth=False` to seedhe kone).
- Road ko `ROADS` list me save karta hai, baad me SDF banane ke liye.
- Road ke saath chhote-chhote kadam leke har point ke aas-paas ek **gola (circle)** grid me "0" se bharta hai.
- `w1` se `w2` tak chaudai dheere badal sakti hai (taper).
- Kam se kam 2 cell chaudi road (`max(1.0, ...)`).

3.5 Boundary, footpath aur left tirchi wall

- **Boundary**: chaaron kinaron par "4".
- **Footpath**: jo ghaas cell road ke 1 cell ke andar hai wo "5" ban jaata hai.
- **Left tirchi wall** grid me nahi, ek **seedhi line ke equation** se bani hai.

```
LW_NX * x + LW_NY * y = LW_C
# (LW_NX, LW_NY) = wall ka normal (perpendicular)
# lw_x(yy) = kisi row par wall ka x
```

Ye wall left badi tirchi road ke saath-saath chalti hai, road + footpath + ek gap ke baad.

4. Building ke numbers

Variable	Value	Matlab
BAY_W	3.0	Facade ke ek khand (bay) ki chaudai
STOREYS	4	Manzil
STOREY_H	1.2	Ek manzil ki unchai (cells)
PARAPET	0.3	Chhat ki mundér
BLD_HT	4*1.2 + 0.3	Total unchai
BLD_W, BLD_D	40, 13	Chaudai aur gehrai
BLD_X0..X1, Y0..Y1	125..165, 131..144	Building rectangle ke corners

Building ek **rectangle box** hai jo beech ki horizontal road ke upar, left road aur vertical road ke beech rakhi gayi hai.

5. Ground texture (bake_ground)

Bake = heavy kaam start me ek baar karke result save kar lena, taaki har frame me na karna pade.

- **PPC = 10**: 1 cell = 10x10 texture pixel.
- **PAD = 30**: map ke bahar bhi 30 cell ghaas.
- **smooth_noise**: random numbers ko bada karke blur karte hain, ghaas me halke-gehre dhabbe aate hain. `box1d` cumulative sum se tez blur karta hai.
- Do noise layers: **L1** (bade dhabbe) aur **L2** (chhote dane).

Texture me **5 channels** hain:

Channel	Kya hai
0, 1, 2	Ghaas ya mitti (wall ke neeche) ka RGB rang
3	Road ki luminance (sirf brightness)
4	Footpath ki luminance (sirf brightness)

Road aur footpath ka **asli rang texture me nahi**, runtime par lagta hai. Isse road ka edge kisi bhi zoom par saaf rehta hai. **TEX_LO** texture ka 5 guna chhota version hai (mipmap jaisa), door ki zameen ke liye, taaki flicker na ho.

6. SDF: code ka sabse smart hissa

SDF = Signed Distance Field. Har pixel par likha hai: "road ke kinare se kitni door?"

- **Negative** = road ke andar
- **0** = bilkul kinara
- **Positive** = road ke bahar

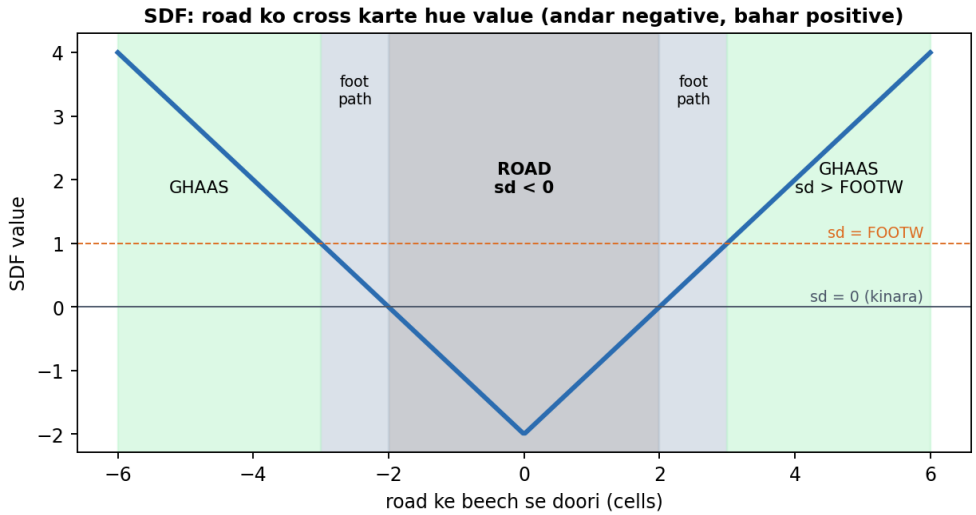


Figure 4: Road ko cross karte hue SDF ki value.

Isse kya milta hai:

- Road ka kinara **bilkul smooth** (grid ke blocky cells nahi dikhte).
- `sd < 0` road, `0 < sd < FOOTW` footpath, baaki ghaas.
- Kerb, edge line aur peeli dashed line sab SDF se nikalti hain.
- **Collision bhi isi se** (`can_walk`): jo dikhta hai wahi roka jaata hai.

Kaise banta hai: har road segment ke liye aas-paas ke pixels par "segment se doori" (`np.hypot`) nikalte hain, road ki aadhi chaudai minus karte hain, aur sab segments ka `minimum` lete hain. Output do channels ka array hai: **sd** aur **dash**.

Runtime par ise **bilinear** sample karte hain (chaaron paas ke pixels ka weighted average), isliye edges har zoom par smooth.

7. Sky panorama

- Ek bahut chauda image (`PANO_W`) jo poore **360 degree** ko cover karta hai.
- **Gradient**: upar gehra neela, neeche halka, horizon par fog ka rang.
- **Sooraj**: gaussian formula se glow image banake paste (wrap ke liye do baar).
- **Badal**: kai random blobs jodke alpha ke saath.
- **Pahadiyan**: `sin()` ki kuch waves jodkar upar-neeche ki line, 2 parat (door wali halki, paas wali gehri).

`draw_sky()` player ke `angle` ke hisaab se panorama ka sahi hissa dikhata hai. Ghumne par aasman bhi ghoomta hai. Seam na dikhe isliye agar hissa kinare se bahar jaaye to panorama dobara blit hota hai.

8. Floor casting (draw_floor)

Har pixel ke liye poochte hain: **"ye screen pixel zameen ke kis (x, y) point ko dekh raha hai?"**

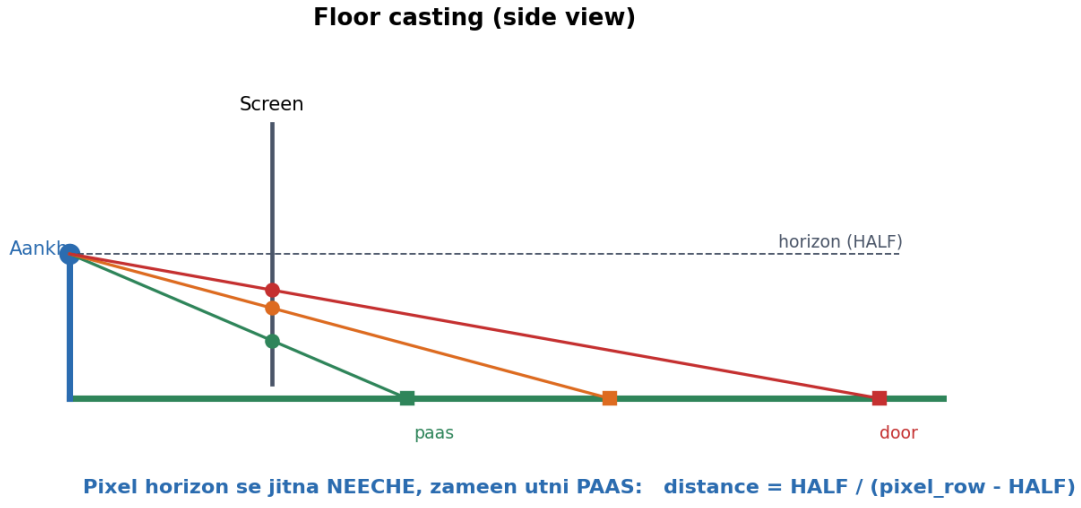


Figure 5: Horizon se jitna neeche pixel, zameen utni paas.

Step by step

Step	Kya hota hai
1. Distance	`distance = HALF / (pixel_row - HALF)`

Step	Kya hota hai
2. Fisheye	`DR = dr / cos(REL_F)`: side ke pixel par asli doori zyada hoti hai
3. World position	`wx = x + cos(angle+REL_F)*DR`, wy same sin se. Numpy se poori screen ek saath, koi loop nahi
4. SDF sample	Bilinear sample, taaki edges smooth rahein
5. Coverage	rc = road kitna, fc = road+footpath kitna. wr, wf, wg = road, footpath, ghaas ka hissa (jod = 1). AA_R door par edge soft karta hai
6. Detail	Paas me noise se dane, footpath par tile lines. Door jaate hi DET1, DET2 se gayab (flicker na ho)
7. Hi/Lo mix	Paas = detailed TEX, door = chhota TEX_LO (MIX_R se blend)
8. Lines	Road ki safed edge line (eb), footpath kerb (kb), peeli dashed line (db)
9. Fog	Door ka rang fog ke rang me mix (FOG_R)
10. Output	Chhoti image ko smoothscale se bada karke screen par

9. Walls: asli raycasting (draw_walls)

Screen ke 300 columns me se har ek ke liye ek kiran player se chalti hai.

Raycasting: player se har column ke liye ek kiran

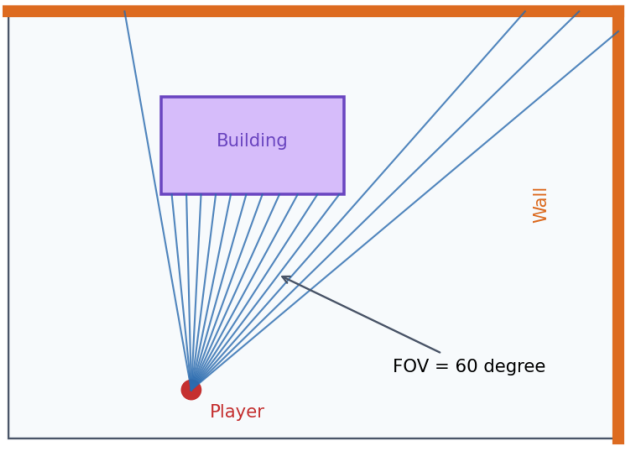


Figure 6: Raycasting ka top view. Har kiran wall ya building se takrakar ruk jaati hai.

Har ray ke liye ye walls check hote hain: **left tirchi wall** (line equation ko ray ke saath solve karke), **right wall** (x = COLS-1), **top wall** (y = TOP), **bottom wall** (y = ROWS-1). `np.minimum` se sabse paas wali chuni jaati hai. `side` batata hai kaunsi taraf ki wall (alag shade ke liye).

Wall ki height

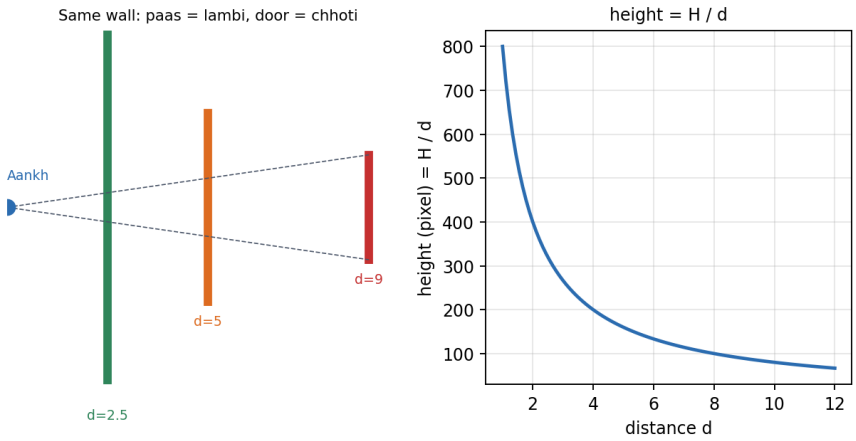


Figure 7: Paas ki wall lambi, door ki chhoti. height = H / d.

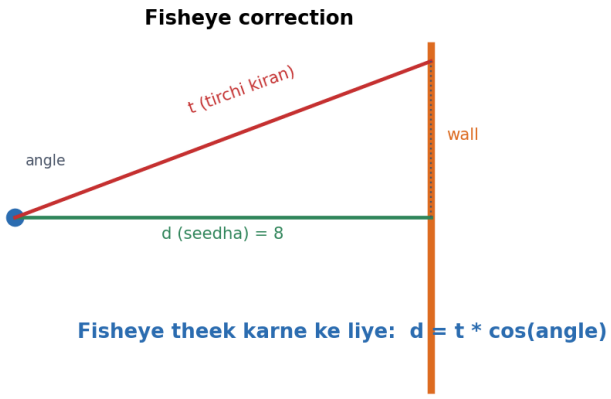


Figure 8: Fisheye hatane ke liye d = t * cos(angle).

```
d      = t * cos(ray_angle)     # fisheye hata ke
height = (H / d) * BOUND_H
```

Texture column: ray jahan takrayi uska decimal hissa u brick texture ka sahi vertical strip chunta hai.

Pre-shaded strips: brick texture ke **16 alag brightness+fog levels** pehle se bane rakhe hain (`WALL_STRIPS`). Runtime par bas sahi level chuno, `scale` karo, `blit` karo.

10. Building

10.1 Facade texture (make_facade)

Ek "bay" ki image numpy se banti hai: deewar ka rang, 4 manzil ke slab, har manzil me 2 khidkiyan (frame, neela sheesha gradient, beech ka mullion), kinare ke column, neeche plinth, upar parapet. Ye texture building ki chaudai par baar-baar repeat hoti hai (`u / BAY_W`).

10.2 building_cast: ray-box (slab method)

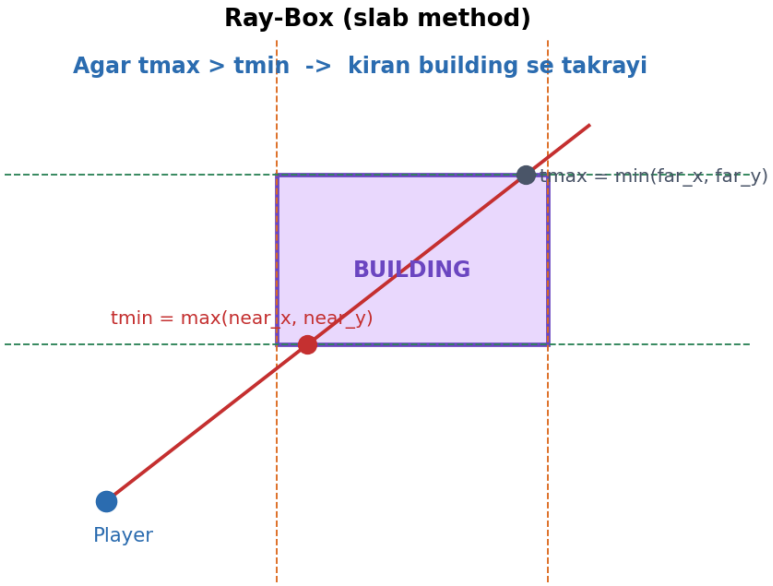


Figure 9: Ray box ko x aur y ki do-do deewaron se kaat-ti hai.

- x ki do deewaron aur y ki do deewaron par takraane ka waqt nikalo.
- `tmin = max(near_x, near_y)`, `tmax = min(far_x, far_y)`.
- Agar `tmax > tmin` to ray box se takrayi.
- Har ray ki depth `BLD_BD` me save hoti hai (ped ki sorting ke liye).

10.3 draw_building

Height = `H/d * BLD_HT`. Paas jaane par building screen se bahar tak jaati hai, isliye sirf dikhne wala hissa crop karke scale karte hain (texture ka bhi utna hi hissa).

11. Ped (billboard sprites)

- **make_tree**: pygame circles se ek ped ki image: tana, shaakhayein, bahut saare hare circles (3 shade). 3 variants.
- **tree_surface**: ek size par scale kiya ped **cache** me rakhta hai (har frame resize nahi).
- **TIN**: andar ke ped (sirf ghaas par, building aur wall se door). **TOUT**: boundary ke bahar ka jungle.
- Footpath aur left wall ke beech sirf 5-6 random ped.

draw_trees

- Player se relative angle aur depth nikalo.
- FOV ke bahar wale chhodo.
- **Door wale pehle** draw karo (sorting).
- Size = `H / depth`.
- Door ke ped par fog.

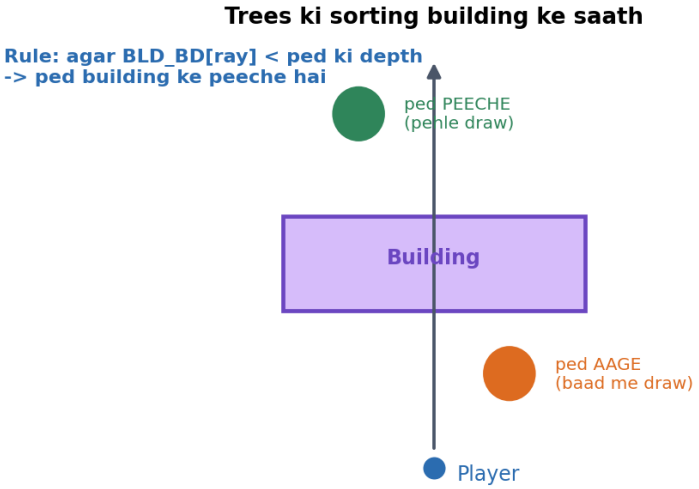


Figure 10: Building ke peeche wale ped pehle, aage wale baad me draw hote hain.

Ped hamesha player ki taraf muh karke hote hain (**billboard**), isliye unhe 3D me ghumane ki zarurat nahi.

12. Vignette aur minimap

- **Vignette**: kinaron par halka kala, taaki cinematic lage.
- **Minimap**: poora MAP ek chhoti image me, building purple. Player laal dot, do peeli lines se FOV.

13. Movement aur collision

- Keys padhke **move** (aage/peeche) aur **strafe** (side) nikalte hain.
- Naya position (nx, ny) banta hai.
- **x aur y alag check** hote hain, isse deewar se takrane par slide karte ho, atakte nahi.
- `can_walk`: player ke 4 corners ka SDF dekhta hai. `SDF >= FOOTW` (ghaas me) to nahi chal sakte.
- **dt**: frame ka time. Slow computer par bhi speed same rahe.

14. Main loop

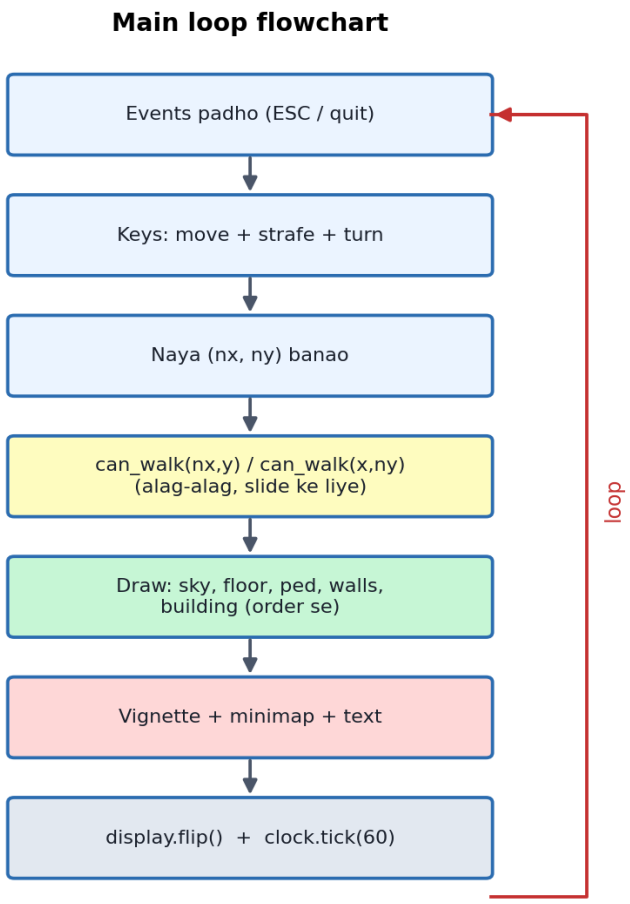


Figure 11: Har frame ka flowchart.

```
events -> movement -> draw_sky -> draw_floor -> draw_trees(TOUT)
-> draw_walls -> building_cast -> draw_trees(TIN, behind)
-> draw_building -> draw_trees(TIN, front) -> vignette -> minimap -> flip
```

15. Sudhar ke sujhav

Building collision (abhi nahi hai)

Abhi `can_walk` sirf SDF dekhta hai, to player building ke andar ghus sakta hai. Fix:

```
def in_building(px, py, r=0.18):
    return (BLD_X0 - r < px < BLD_X1 + r and
            BLD_Y0 - r < py < BLD_Y1 + r)

def can_walk(px, py, r=0.18):
    if in_building(px, py):
        return False
    ...  # baaki purana code
```

Performance

`draw_walls` aur `draw_building` me har column par `pygame.transform.scale` hota hai (300 baar per frame). Ye Python loop slow hai. Options: RAYS kam karo, `FLOOR_SCALE = 3` karo. Bade campus (K=3, EXTRA_COLS=60) par texture memory bhi badi hoti hai (`GH x GW x 5` bytes) aur start thoda slow hota hai.

Aage ke idea

- Building me darwaza, board aur entrance
- Aur buildings aur roads
- Day/night cycle
- NPCs aur sound

16. Chalane ka tareeka

```
pip install pygame numpy
python campus.py
```

Key	Kaam
W / S / A / D ya arrows	Chalo
Left / Right	Ghumo
Shift	Tez chalo
ESC	Band

17. 30 second pitch

"Ye MITS campus ka 3D walkthrough hai jo bina 3D engine ke, sirf pygame aur numpy se bana hai. Campus ka 2D map hai. Zameen ke har pixel ke liye maths se nikalte hain wo map ke kis point ko dekh raha hai (floor casting). Deewaron ke liye har column me ek kiran bhejte hain (raycasting). Roads ke smooth kinare ek distance field se aate hain. Building ek box hai jis par khidki wali texture lagi hai, aur ped chhoti images hain jo door-se-paas order me draw hote hain."

Glossary

Shabd	Matlab
Raycasting	Har column ke liye ek kiran, wall tak ki doori se height
Floor casting	Har zameen ke pixel ke liye world position nikalna
SDF	Har point par "kinare se doori": smooth edges aur collision
Bake	Heavy kaam ek baar start me, runtime par sirf sample
Painter's algorithm	Door wali cheez pehle, paas wali baad me
Billboard	Sprite jo hamesha player ki taraf muh kiye rehta hai
Fisheye	Side ki kirano ka lamba rasta, d = t * cos(angle) se theek
FOV	Field of view, player ka dekhne ka angle
Mipmap	Door ke liye chhota texture, flicker rokne ke liye